Bob, verified: pitch

Team: Alan Finney, 3Dogs Nexus · Track: Modernize What Matters · AngelHack x IBM, Building with IBM Bob

What we accomplished

We took a nine-year-old Flask to-do service that no longer ran, handed it to IBM Bob, and got it running on Python 3.12 in three steps for 2.25 credits, with every step proven rather than claimed. We did it by building a verification layer around Bob: after each step an independent gate runs the step's own check and the app's smoke checks, then runs the same checks on a deliberately broken copy; if the checks still pass on broken code they are too weak and the step is refused. All three steps were accepted, every broken copy failed every check, the cost ledger matched Bob's own task store to four decimals, and a zero-cost re-verify from the live dashboard re-ran everything and accepted it again. Bob wrote every implementation module of the tool from tests the team wrote first.

Demo video: https://youtu.be/qAyUG-xH8RY. Live dashboard: https://18-216-235-168.sslip.io/. Code: https://github.com/DKJager/bob-verified

1. Team and introduction

Alan Finney, founder of 3Dogs Nexus, a startup that sells consulting on problems advanced AI can solve. 3Dogs Nexus builds for clients and also advises clients who run their own questions through its tools. Team of one, working with IBM Bob as its engineering partner: Bob wrote every implementation module of this tool except the ported task runner, and carried out the modernization shown below.

2. The problem

AI coding agents now change legacy code faster than a team can review it. The failure is not the code Bob writes; it is trusting a step that says "done". Checks that always pass, a cost ledger that drifts from the real bill, and no way to see afterwards what was verified. An earlier modernization attempt by the team (15 Oct, a larger Django app, not part of this entry) recorded 15.42 credits in its own ledger while Bob's task store showed 39.92.

3. The solution

A thin layer around IBM Bob in five stages: plan (Bob in plan mode, at most 3 steps, one verification command each), step (Bob in agent mode, one step at a time), gate (the step's command, the app's smoke checks, and a negative control on a deliberately broken copy; checks that cannot fail are refused as CHECKS_TOO_WEAK), ledger (per-step credits reconciled against Bob's own task store), dashboard (read-only, with a re-verify button that re-runs everything at zero cost and logs it with timestamps).

4. What was built (16 to 18 Oct 2026)

Python 3.12, standard library only: gate, ledger reconciliation, plan-to-runner wiring, dashboard and re-verify, fetch script, deploy files. 99 automated tests, including a negative test (a step that claims success but is broken is rejected) and an end-to-end test with a fake Bob. Tests were written first by the team; IBM Bob wrote the implementations in 14 small tasks for 4.22 credits. The task runner was ported from a version written on 15 Oct by the team, not by Bob.

5. The recorded run

Target: simple_api, a 2017 Flask, Flask-RESTful and SQLAlchemy todo API (MIT, 107 lines of Python), fetched at a pinned commit. It does not start on current Flask because of a removed import.

Step	Result	Gate	Negative control	Credits
plan (Bob, plan mode)	3 steps	plan validated	n/a	0.19
step_01 make it run on Python 3.12	accepted	ACCEPT	broken copy failed 4 of 4 checks	0.38
step_02 SQLAlchemy 2 and a missing-id bug	accepted	ACCEPT	failed 4 of 4	1.38
step_03 add a pytest suite (9 tests)	accepted	ACCEPT	failed 4 of 4	0.29
Total	3 of 3 steps accepted			2.25

Ledger 2.2479 against Bob's own store 2.2479: MATCH. A deliberately re-broken tree was rejected by the gate (4 checks failed). Re-verify from the dashboard re-ran all checks against the modernized target in 94 seconds with no AI call: all ACCEPT. Live demo: https://18-216-235-168.sslip.io/ (re-verify limited to one run at a time, 6 per hour per visitor, 120 s hard stop).

6. Against the judging criteria

- Tool usage and depth of adoption: IBM Bob is used as builder, planner and implementer. 18 Bob tasks in total: 14 to build the tool, 1 in plan mode, 3 in agent mode on the target. Every Bob commit carries Bob-Task and Bob-Trace trailers. Cost is reconciled to Bob's own store with a difference of 0.0000.
- Innovation and use case: the negative control. A check is only trusted if it fails on a broken copy, so "the tests passed" means something. Cost reconciliation and zero-cost re-verification make AI-assisted modernization auditable.
- Functionality and technical execution: 99 tests passing; a real end-to-end run on a real legacy app; all three steps accepted, every control caught; real re-verify with a timestamped log; deployable with a systemd unit or Docker.
- Presentation: a live dashboard anyone can open, a README that states what is and is not the contribution, a 3-minute script.

7. Roadmap and key learnings

Roadmap: more targets and languages, parallel steps with merge checks, a hosted service where a team uploads a repository and receives a verified modernization with its evidence, and per-step spending limits that stop a run automatically.

Key learnings: small tasks with exact tests cost a fraction of big ones (14 tool-building tasks for 4.22 credits, against long multi-step tasks that hit turn limits earlier); a ledger must be reconciled against the tool's own store; a check is only as good as its ability to fail; the human's best contribution is the contract (tests and specifications), not the code.